

RELATÓRIO – PBJL3 FUNÇÃO HASH

Luan Estevinho, João Davi, Laura Martins

O relatório a seguir apresenta uma análise branda de 2 funções hashes distintas operando em uma tabela hash, está sendo utilizado como base uma lista com 5mil nomes. O relatório visa comparar o desempenho das duas funções hashes, avaliar o comportamento das colisões e a eficiência das operações de busca, inserção e remoção.

1.Estrutura

O código está em java e possui as seguintes classes:

- **HashGeral:** Classe abstrata que define os comportamentos padrões da tabela hash, contém os métodos de inserção, busca, remoção, redimensionamento e contagem de colisões.
- **FuncHash1:** Classe que herda a HashGeral e implementa uma função hash simples, soma os valores ASCII dos caracteres da chave
- **FuncHash2:** Classe que herda a HashGeral e implementa uma função hash polinomial, que multiplica o valor acumulado por 31 a cada caractere da string.
- **Main:** Responsável pela leitura do arquivo contendo os 5mil nomes, realização das operações dispostas, medição do tempo e exibição dos resultados.

2.Métodos Hash Implementados

Função 1 – Soma Simples:

Essa função soma os valores ASCII de todos os caracteres da string e aplica (%) com o tamanho da tabela para encaixar o resultado da soma em um índice da tabela.

Resumo: $\sum \text{Ascii}(\text{letra}) \% \text{tamanho da tabela}$

Função 2 – Polinomial:

Essa função usa um calculo polinomial, ele multiplica o valor acumulado por 31 a cada iteração, faz isso para gerar uma dispersão maior na tabela, por conta da variabilidade nos resultados.

Resumo: $(\text{hash} * 31 + \text{Ascii}(\text{letra})) \% \text{tamanho}$

Aqui também usa o % tamanho para se encaixar em um índice da tabela.

3.Tratamento de Colisões

O método de tratamento que escolhi foi a sondagem linear (vista em aula). Quando ocorre uma colisão, ou seja, 2 chaves tentar entrar no mesmo índice, o algoritmo vai avançar sequencialmente para encontrar uma posição vazia, e faz isso até encontrar.

Resumo: $\text{posição} = (\text{posição} + 1) \% \text{tamanho}$

Quando não encontra um elemento null ou "vazio" (mais abaixo vem a explicação do "vazio") ele soma 1 na posição e no final faz % tamanho para garantir que a posição não de maior que o tamanho da tabela

4. Redimensionamento da Tabela (Rehashing)

No meu código o redimensionamento funciona da seguinte maneira: Sempre que o fator de carga (numero de elementos / tamanho da tabela) passa de 0,75 (75%) a tabela é redimensionada para o dobro do tamanho anterior. Após isso, todos os elementos que já estavam na tabela são reinseridos, fazendo o Rehashing com base no novo tamanho.

Resumo: 32 -> 64 -> 128 -> 256 -> 512 -> 1024 -> 2048 -> 4096 -> 8192

No código ele chegou ate o tamanho de 8192, mostrando que o redimensionamento foi corretamente acionado.

4.5 Análise da Distribuição de Colisões

Aqui foram analisados os padrões de colisões obtidos após a inserção dos 5000 nomes em cada uma das tabelas hash.

Tabela 1 – Função hash de soma simples:

O comportamento da função teve um padrão bem definido e previsível, que mostra as limitações dessa função.

FAIXA DE INDICE	COMPORTAMENTO
0 - 1500	Aumento gradual até cerca de 1900 colisões por índice
1500 – 3350	Região ficou estável, mantendo uma alta taxa de colisões por índice
3350 – 5250	Decaimento gradual, reduzindo cerca de uma colisão por índice
5250 - 8191	Região completamente vazia, com 0 colisões por índice

As somas dos valores da tabela ASCII dos nomes acabam tendo valores bem próximos, se concentrando nos índices iniciais. Isso acabou gerando um crescimento rápido de colisões nessa faixa.

A função se manteve mapeando algumas diferentes chaves para intervalos bem próximos, o que continuou saturando as mesmas posições da tabela.

Como efeito da sondagem linear, que acaba empurrando os elementos pra frente quando há colisões, Acabou gerando esse decaimento progressivo, tipo uma “cauda”

E nenhum valor de hash atingiu a faixa mais alta da tabela, mostrando que a função não gerou índices altos o suficiente e desperdiçou boa parte da tabela.

Tabela 2 – Função hash polinomial:

Essa Segunda função usou a multiplicação polinomial, onde cada caractere é multiplicado por 31 antes da adição do próximo caractere.

A análise dessa distribuição foi completamente diferente da função 1:

- As colisões foram muito baixas e bem distribuídas por toda a tabela
- A variação entre os índices ficou entre 0 – 5 colisões por posição sem uma formação relevante de picos ou regiões com aglomerado de colisões
- Nenhum agrupamento excessivo foi observado e praticamente todas as posições foram ocupadas de forma equilibrada

5. Resultados

Redimensionamento:

As 2 funções passaram pelo mesmo numero de redimensionamentos durante a execução, ficando no final com um tamanho de 8192 posições.

Tempo de inserção:

FUNÇÃO	TEMPO
SOMA SIMPLES	≈ 138.528.500 ns
POLINOMIAL	≈ 62.206.500 ns

Portanto, entende-se que a função de hash polinomial foi praticamente 2 vezes mais rápida que a de soma simples, devido a menor quantidade de colisões e menor necessidade de sondagem linear

Tempo de Busca:

FUNÇÃO	PALAVRA	RESULTADO	TEMPO(NS)
SOMA SIMPLES	Amanda	Encontrada	37.700ns
POLINOMIAL	Amanda	Encontrada	3.700ns

O tempo mostra que a função polinomial foi pouco mais de 10x mais rápida que a soma simples.

Tempo de Remoção:

FUNÇÃO	PALAVRA	TEMPO(NS)
SOMA SIMPLES	Amanda	42.600ns
POLINOMIAL	Amanda	3.700ns

Novamente a função polinomial mostra um desempenho superior a de soma simples

Colisões Totais:

FUNÇÃO	COLISÕES
SOMA SIMPLES	10.686.910
POLINOMIAL	12.185

A diferença entre as funções é extremamente significativa, a função de soma simples gerou quase 900x mais colisões que a polinomial.

6. Análise dos Resultados

Os resultados mostrados acima mostram que:

- A função hash polinomial é muito mais eficiente por conta do seu método de multiplicação, que como já comentado, gera uma variabilidade de chaves muito maior, não fazendo com que múltiplos elementos diferentes caiam no mesmo índice.
- A função hash soma simples teve um desempenho insatisfatório comparado com a polinomial principalmente pela alta taxa de colisões. Que acontece por conta da semelhança entre os valores da ACSII das letras da palavra.

Ou seja, na função de soma simples, a string "ANA" e a string "NAA" vão resultar em valores iguais, já na função polinomial, essas strings resultariam em valores completamente diferentes. Portanto, como a soma simples só considera o valor final, se a palavra tiver as mesmas letras em ordem diferente o resultado é o mesmo. Na polinomial ele multiplica o valor anterior por 31 a cada passo, então a troca de posições da letra altera completamente o resultado.

Exemplo prático:

Soma simples:

$$\text{ANA} = 65 + 78 + 65 = 208 \% 32 = 16$$

$$\text{NAA} = 78 + 65 + 65 = 208 \% 32 = 16$$

Polinomial:

Início hash = 7

$$\text{ANA} = (((7 * 31) + 65) * 31 + 78) * 31 + 65 = 263.925 \% 32 = 21$$

$$\text{NAA} = (((7 * 31) + 78) * 31 + 65) * 31 + 65 = 285.575 \% 32 = 7$$

7. Conclusão

A função 1 se baseia na soma simples dos valores ASCII, e mostrou um número muito elevado de colisões, além de tempos de execução maiores em todas as operações. Isso aconteceu porque essa função não considera a ordem dos caracteres, tratando palavras diferentes formadas pelas mesmas letras como iguais, como é o exemplo que dei acima: "ANA" e "NAA" que resultam exatamente no mesmo valor hash mesmo sendo

palavras diferentes. Isso faz com que gere um aglomerado de dados em poucas posições da tabela, assim reduzindo a eficiência geral do código.

Já a função 2, que usa um calculo polinomial para fazer os hashes, leva em conta a posição de cada caractere na string, pois multiplica o valor acumulado por 31 a cada iteração. Isso faz com que pequenas mudanças na palavra gerem resultados completamente diferentes, assim trazendo uma distribuição bem mais uniforme e variada.

Falando de desempenho, a função hash polinomial foi mais de 2x mais rápida na inserção e umas 10x mais rápida na busca e remoção. Portanto entende-se que a função hash polinomial foi altamente superior tanto em tempo quanto em distribuição.

Comparação entre as funções:

CRITÉRIO	SOMA SIMPLES	POLINOMIAL
ORDEM DOS CARACTERES	Não	Sim
DISTRIBUIÇÃO DOS INDICES	Concentrada nas regiões do início	Uniforme pela tabela
PICOS DE COLISÃO	Alta (até 1900 por índice)	Ausente
ESPAÇO OCIOSO NA TABELA	Alto (quase metade da tabela sem uso)	Minimo
TOTAL DE COLISÕES	10.686.910	12.185
TEMPO MÉDIO DE INSERÇÃO	138.528.500 ns	62.206.500 ns
TEMPO DE BUSCA	37.700 ns	3.700 ns
TEMPO DE REMOÇÃO	42.600 ns	3.700 ns

